Extra Credit: Link-Quality Routing

The idea

Hop count thinks every link is equally good. So it will pick a short path with bad links over a longer path with good links. We wanted bad links to cost more so routing avoids them.

How we measure a link

- Neighbor discovery already sends a PING every 10 s and waits for a reply. We just keep track of whether each neighbor replied.
- For each neighbor we remember the last 16 rounds (1 = replied, 0 = didn't).
- A reply needs the link to work both ways, so this checks both directions.
- No extra packets are needed.

Link cost

- 14–16 replies out of 16 → cost 1
- 12–13 → cost 2
- 8–11 → cost 3
- less than 8 → cost 4

The cost only changes if the new value shows up 3 rounds in a row, so one bad round doesn't change it.

How routing uses it

- Before: cost through a neighbor = their cost + 1
- Now: cost through a neighbor = their cost + the link cost
- Everything else is the same: INFINITY = 16, poison reverse, timeouts.
- It can't create loops because every link costs at least 1, so a loop always costs more.
- Downside: bad links cost more, so a path made of bad links hits 16 sooner.
- We added a switch in `dv.h` (`USE_LQ_METRIC`) so we can test both versions with the same code.

Test

- Two paths from node 1 to node 4:
  - short path 1-2-4 with weak links (-75)
  - long path 1-3-5-4 with strong links (-54)
- Same noise file (`meyer-heavy.txt`) and same random seed for both versions
- Wait 300 s, then send 50 pings from 1 to 4

Results

Run	Hop count	Link quality
1	31 / 50	32 / 50
2	33 / 50	39 / 50

Run	Hop count	Link quality
3	30 / 50	35 / 50
Average	31.3	35.3

Link quality got about 13% more pings back. Hop count used the bad short path every time. Link quality noticed the link from 1 to 2 was bad and sent most pings through 3 instead.

Problems we ran into

- Our first try only remembered 8 rounds and did worse than hop count (17 vs 27). One lost reply was enough to change the cost, so routes kept switching back and forth. Remembering 16 rounds and waiting 3 rounds before changing the cost fixed most of it.
- Routes still switched more than with hop count (10 times vs 2), because the weak links were right on the edge between cost 2 and 3. Next time we would only switch paths if the new one is clearly better.
- Results were a little different every run because of random noise, so we used a fixed seed and compared a few runs.